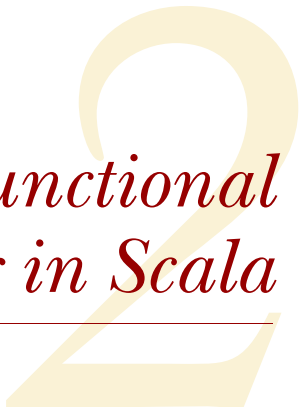


Getting started with functional programming in Scala



Now that we have committed to using only pure functions, a question naturally emerges: how do we write even the simplest of programs? Most of us are used to thinking of programs as sequences of instructions that are executed in order, where each instruction has some kind of effect. In this chapter, we'll begin learning how to write programs in the Scala language just by combining pure functions.

This chapter is mainly intended for those readers who are new to Scala, to functional programming, or both. Immersion is an effective method for learning a foreign language, so we'll just dive in. The only way for Scala code to look familiar and not foreign is if we look at a lot of Scala code. We've already seen some in the first chapter. In this chapter, we'll start by looking at a small but complete program. We'll then break it down piece by piece to examine what it does in some detail, in order to understand the basics of the Scala language and its syntax. Our goal in this book is to teach functional programming, but we'll use Scala as our vehicle, and need to know enough of the Scala language and its syntax to get going.

Once we've covered some of the basic elements of the Scala language, we'll then introduce some of the basic techniques for how to write functional programs. We'll discuss how to write loops using *tail recursive functions*, and we'll introduce *higher-order functions* (HOFs). HOFs are functions that take other functions as arguments and may themselves return functions as their output. We'll also look at some examples of *polymorphic* HOFs where we use types to guide us toward an implementation.

There's a lot of new material in this chapter. Some of the material related to HOFs may be brain-bending if you have a lot of experience programming in a language *without* the ability to pass functions around like that. Remember, it's not crucial that you internalize every single concept in this chapter, or solve every exercise.